

- (*) CLAUSE :- (1) WHERE CLAUSE. (4) ORDER BY CLAUSE.
(2) GROUP BY CLAUSE. (5) LIMIT
(3) HAVING CLAUSE. (6) OFFSET.

(2) GROUP BY CLAUSE :- GROUP BY clause groups rows sharing the same values in specified columns into summary rows. It is always used with aggregate functions.

Syntax: SELECT non-aggregate column, aggregate function(column)
FROM table-name
GROUP BY non-aggregate column;

NOTE: All non aggregate column present in select should be used in GROUP BY clause.

(3) HAVING :- HAVING filters groups after GROUP BY aggregation. Unlike WHERE (which filters rows), HAVING filters groups & can use aggregate functions.

Syntax: SELECT non-aggregate column, aggregate function(column)
FROM table-name.

GROUP BY non-aggregate column

HAVING aggregate-function <condition>;

Difference b/w WHERE & HAVING :-

WHERE	HAVING
• Filters rows. • Used before GROUP BY. • Cannot use aggregate functions.	• Filters groups. • Used after GROUP BY • Can use aggregate functions.

(4) ORDER BY :- This sorts the result set by one or more columns in ascending (ASC, default) or descending (DESC) order.

Syntax: SELECT * FROM table-name
ORDER BY column-name;

(5) LIMIT :- This restricts the number of rows returned by a query starting from top.

Syntax: SELECT * FROM table-name
LIMIT value;

(6) OFFSET :- This skips a specified number of rows before starting to return results.

Syntax: SELECT * FROM table-name
LIMIT value
OFFSET value;

(*) Subqueries :- A query inside another query.

Syntax: SELECT column-name (op data)
FROM table-name
<Where condition> (select column-name
(input data) FROM table-name);

> When do we have to go for sub-queries?

If we want to fetch the output from one table based on the input of another table.

> Execution Process:-

> First the inner query get executed.

> Based on the inner query output, the outer query get executed.

> Types of Subqueries:-

(i) Single-row Subquery.

(ii) Multi-row Subquery.

(iii) Co-related Subquery.

(i) Single-row Subquery $\Rightarrow$ If the inner query is displaying only-one output, then it is called as single-row Subquery.

Incase of single-row subquery, we can use the "=" operator between the outer query & inner query.

(ii) Multi-row Subquery $\Rightarrow$

> If the inner queries return more than one output, then it is called multi-row Subquery.

> Incase of multi-row subquery we have to use the 'IN' operator between outer query & inner query.

(iii) Co-related Subquery $\Rightarrow$ A Correlated Subquery is a Subquery that uses values from the outer query.

Example: Find Employees Earning more than their Department Average Salary.

```

SELECT emp_name, department, salary
FROM employee e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employee e2
    WHERE e1.department = e2.department
);

```

= $\Rightarrow$ Fetch single record.

IN $\Rightarrow$ Fetch multiple records.

(*) VIEWS:- A view is a virtual table created from one or more tables using a query. It does not store data permanently. It shows data from the original table whenever accessed.

- > Hides Complex Queries.
- > Improve Security.
- > Simplify data access.
- > Restrict access to specific columns.

Syntax:- ~~Create view~~;

1. Create view:- CREATE VIEW view_name AS
 SELECT column1, column2
 FROM table_name
 WHERE condition;

2. Access view:- SELECT * FROM view_name;

3. Drop view:- DROP VIEW view_name;

Difference between view and Table:-

VIEW	TABLE
> Virtual table	> Physical table

- > Stores query only.
- > Less memory.
- > Data comes from base table.

- > Stores actual data.
- > More ~~memory~~ memory.
- > data stored directly.

(*) Index:- It is used to search the data in faster way. It is used to increase the database performance. We can create predefined index & our own index as well.

INDEX

Clustered Index (0)
⇒ Primary Key.

Non-clustered Index (1)
⇒ Non-Primary Key.

Syntax:-

1. Create Index: CREATE INDEX index-name -> idx_name
ON table-name (column-name);

2. View Index: SHOW INDEX FROM table-name;

3. Delete Index: DROP INDEX index-name
ON table-name;

Advantage	Disadvantage
> Faster Retrieval	> Uses extra storage.
> Improves WHERE performance	> Slows insert/update/delete.